Développer pour le Cloud

INFO - M2 Expert en développement web

CI/CD

Qu’est-ce qu’une pipeline ?

CI/CD

1. CI/CD
2. Variables et secrets
3. Analyse d’une pipeline
4. Environnements
5. Versionning
6. Workflow complet

CI (Continuous Integration)

Vérifier automatiquement que le code reste valide à chaque changement.

Permet d’éviter les différences de fonctionnement entre environnements

CI (Continuous Integration)

Vérifier automatiquement que le code reste valide à chaque changement.

Permet d’éviter les différences de fonctionnement entre environnements

Sans CI :

- tests exécutés au hasard
- builds non reproductibles
- erreurs humaines (tag, push, oubli d'étape)

Avec CI :

- une machine neutre (runner) répète les mêmes étapes
- qualité testée avant merge (PR gate)

CD (Continuous Delivery / Deployment)

Delivery : l'artefact est prêt à être déployé à tout moment (déploiement souvent manuel).

Deployment : le déploiement est automatique.

La règle d'or : **Une CI qui teste**

Minimum vital :

- des tests rapides, qui passent en quelques secondes
- au moins un test "contrat HTTP" (ex: /health)
- au moins un test de validation (400) pour empêcher les régressions de validation

Pipeline minimale

```yaml
name: CI - Build & Push Docker image

on:
  push:
    branches: ["main"]

env:
  IMAGE_NAME: yourdockerhubuser/notes-api

jobs:
  build-test-push:
    runs-on: ubuntu-latest

    steps:

        [...]
```

Variables d’environnement : lesquelles, et où ?

En général, `.env` est dans `.gitignore`, donc **il n’existe pas dans le runner CI**.

Il faut donc identifier les variables nécessaire à la pipeline et faire la différence entre les variables :

- pour faire tourner l’app en prod (runtime)
- pour la pipeline (build/CI)

Variables pour faire tourner l'app en prod (runtime)

Exemples : `DATABASE_URL`, `PORT`, etc.

➡️ Pas nécessaire pour build/push l'image.

➡️ Elles seront injectées au moment du `docker run`, en cloud (ou en compose).

Variables pour le pipeline (build/CI)

Exemples : `IMAGE_NAME`, `TAG`, etc.

➡️ Oui, on les met dans le YAML (ou GitHub Variables).

Variables ≠ Secrets

Quand on **configure des “variables” côté GitHub**, on distingue deux catégories :

Secrets (sensibles)

- identifiants / tokens Docker Hub
- mots de passe DB

Variables (non sensibles)

- nom de l’image (`IMAGE_NAME`)
- registry (`docker.io`)
- nom du repo

Variables dans le YAML

On déclare les variables dans un tag `env`

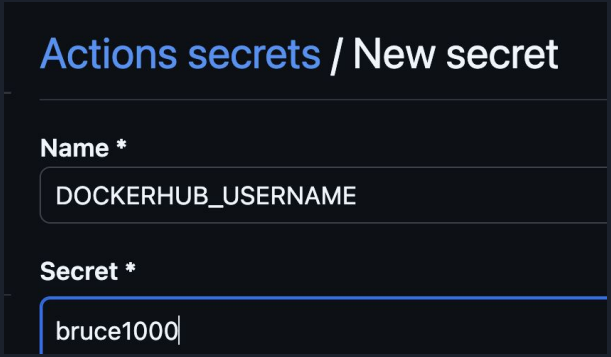
```
env:
  IMAGE_NAME: <yourdockerhubuser>/notes-api
```

On peut ensui utiliser les variables `env.VAR_NAME`

```
- name: Build and push
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ${{ env.IMAGE_NAME }}:latest
```

Secrets dans le YAML

On déclare les variables depuis l'interface GitHub.



On peut ensui utiliser les variables `secrets.VAR_NAME`

```
- name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
```

Pipeline conceptuelle

Git push
↓
Checkout code
↓
Install dependencies
↓
Run tests
↓
Docker login
↓
Build image
↓
Push image

CI ?

CD ?

Pipeline conceptuelle

Git push
↓
Checkout code
↓
Install dependencies
↓
Run tests
↓
Docker login
↓
Build image
↓
Push image

CI

- Checkout code
- Install dependencies
- Run tests

CD

- Login Docker Hub
- Build & Push image

Pipeline minimale

```
# 1️⃣ Récupérer le code
- name: Checkout code
uses: actions/checkout@v4

# 2️⃣ Préparer l'environnement Node
- name: Set up Node
uses: actions/setup-node@v4
with:
node-version: 20
cache: "npm"

# 3️⃣ Installer les dépendances
- name: Install dependencies
run: npm ci
```

Pipeline minimale

```
# 1️⃣ Récupérer le code
- name: Checkout code
uses: actions/checkout@v4

# 2️⃣ Préparer l'environnement Node
- name: Set up Node
uses: actions/setup-node@v4
with:
node-version: 20
cache: "npm"

# 3️⃣ Installer les dépendances
- name: Install dependencies
run: npm ci
```

`checkout` = “ramène le repo dans la machine du pipeline”

Pipeline minimale

```
# 1️⃣ Récupérer le code
- name: Checkout code
uses: actions/checkout@v4

# 2️⃣ Préparer l'environnement Node
- name: Set up Node
uses: actions/setup-node@v4
with:
node-version: 20
cache: "npm"

# 3️⃣ Installer les dépendances
- name: Install dependencies
run: npm ci
```

`checkout` = “ramène le repo dans la machine du pipeline”

installe Node sur le runner.

`cache: "npm"` accélère en gardant le téléchargement des dépendances entre runs.

Pipeline minimale

```
# 1️⃣ Récupérer le code
- name: Checkout code
uses: actions/checkout@v4

# 2️⃣ Préparer l'environnement Node
- name: Set up Node
uses: actions/setup-node@v4
with:
node-version: 20
cache: "npm"

# 3️⃣ Installer les dépendances
- name: Install dependencies
run: npm ci
```

`checkout` = “ramène le repo dans la machine du pipeline”

installe Node sur le runner.

`cache: "npm"` accélère en gardant le téléchargement des dépendances entre runs.

`npm ci` = “je veux un build déterministe”

Pipeline minimale

```
# 4️⃣ Lancer les tests
- name: Run tests
    run: npm test

# 5️⃣ Authentification au registry
- name: Log in to Docker Hub
    uses: docker/login-action@v3
    with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}

# 6️⃣ Build + Push de l'image
- name: Build and push
    uses: docker/build-push-action@v6
    with:
    context: .
    push: true
    tags: ${{ env.IMAGE_NAME }}:latest
```

Déployer sur plusieurs environnements

On identifie 3 environnements :

- **Local**
- **Production** (ou **prod**)

l'environnement "réel". Les vrais utilisateurs, les vraies données, la vraie facture cloud, la vraie responsabilité.

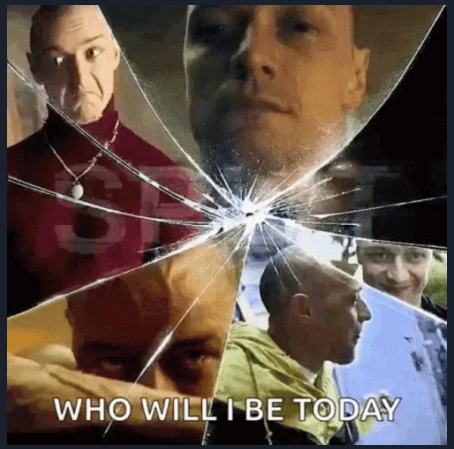
- **Staging** (optionel)

une **réplique aussi proche que possible de prod**, mais utilisée pour valider avant de livrer aux utilisateurs.

Déployer sur plusieurs environnements

La même image Docker tourne sur les 3 environnements, mais :

- configuration différente
- secrets différents
- ressources différentes



Déployer sur plusieurs environnements

La même image Docker tourne sur les 3 environnements, mais :

- configuration différente
- secrets différents
- ressources différentes

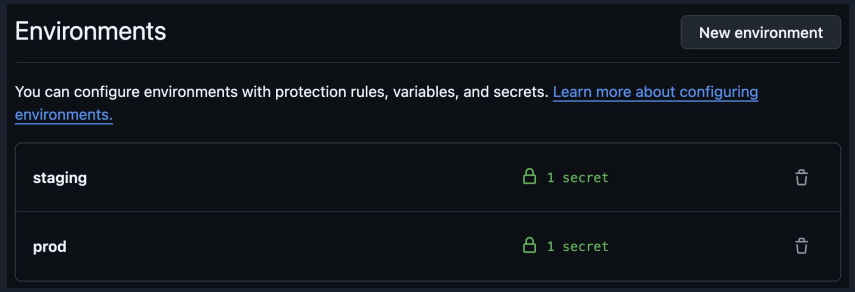
Exemples concrets de différences typiques :

- **Base de données**
- **Clés/API**
- **Observabilité**
- **Règles de déploiement**

Configurer les variables selon l'environnement

Local : `.env` (non commité)

GitHub a une feature "Environments" (au niveau repo) : `staging`, `production`, etc.



Chaque environnement peut avoir :

- ses secrets
- ses variables
- éventuellement une validation manuelle

Déploiement de versions

Si l'on reprend l'étape 6️⃣ de notre pipeline (Build + push de l'image)

```
- name: Build and push
    uses: docker/build-push-action@v6
    with:
    context: .
    push: true
    tags: ${{ env.IMAGE_NAME }}:latest
```

On remarque que l'on déploie une image avec le tag `latest`

En quoi déployer `<mon-image>:latest` peut-il être problématique ?

Déploiement de versions - Tag latest

Entre deux déploiements, le tag `latest` “bouge”.

- `latest` ne contient aucune information de version
- Il ne garantit aucune stabilité
- Il peut pointer vers des contenus différents à différents moments

Déploiement de versions - Tag latest

Entre deux déploiements, le tag `latest` “bouge”.

- `latest` ne contient aucune information de version
- Il ne garantit aucune stabilité
- Il peut pointer vers des contenus différents à différents moments

👉 Ce n’est **pas une version**. C’est **un alias pratique** pour le développement.

Déploiement de versions - Tag latest

Conséquence en cloud :

- Instance 1 télécharge latest à 10h
- Instance 2 télécharge latest à 10h05

Si entre-temps une nouvelle image a été poussé, alors les **deux instances ne tournent pas le même code**.

Déploiement de versions - Digest

Une image Docker est identifiée en réalité par un **digest** :
`sha256:9f1b0a4c...`

Il est calculé à partir du contenu exact de l'image (les layers, le filesystem, etc.)

Si un seul bit change → le digest change.

Digest = identité cryptographique **immuable**

- Même contenu → même digest
- Contenu différent → digest différent

Déploiement de versions - Digest

Un tag pointe vers un digest. Si l'on fait `docker push <image>:v1.0.0`

Docker Hub enregistre `:v1.0.0 →sha256:XYZ`

Re-publier une autre image avec le même tag v1.0.0 casse l'immutabilité logique de la version.

👉 **Une vraie version ne doit jamais être republiée.**

Déploiement de versions - Stratégie de tags

Niveau 1

1. `:latest` → alias mutable (pratique pour tester rapidement)
2. `:<git-sha>` → version immuable et traçable

Dans la pipeline :

```
- name: Build and push
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ${{ env.IMAGE_NAME }}:latest
      ${{ env.IMAGE_NAME }}:${{ github.sha
}}
```

Déploiement de versions - Stratégie de tags

Niveau 2

Nous adoptons SemVer : *vMAJOR.MINOR.PATCH* (ex: v1.2.3)

- MAJOR : breaking change
- MINOR : feature backward-compatible
- PATCH : bugfix

Une release est déclarée dans Git par un tag.

```
git tag v1.0.0
git push origin v1.0.0
```

Déploiement de versions - Stratégie de tags

Stratégie complète

Sur push main :

- latest
- sha

Sur tag Git vX.Y.Z :

- vX.Y.Z

Déploiement de versions - Stratégie de tags

Stratégie complète

Sur push main :

- latest
- sha

Sur tag Git vX.Y.Z :

- vX.Y.Z

⚠️ Point d'attention
Une version v1.2.0 ne doit jamais être reconstruite. Si bug : → v1.2.1

Déploiement de versions - Traceability

Avec notre stratégie, on peut répondre à "Qu'est-ce qui tourne ?" :

- Quel commit ?
- Quel run CI ?
- Quelle version produit ?
- Quelle image (digest) ?

Workflow complet - 1 CI sur PR

Objectif : “Ne pas merger sur `main` si ça casse”.

- Trigger : `pull_request` vers `main`
- Steps : `npm ci` + `npm test` (éventuellement `lint` etc ..)

Pas de `push` ni de `deploy`

✅ Qualité et revue de code
✅ Gate avant merge

Workflow complet - 2 CI sur main CD sur staging

Objectif : “`main` produit un artefact et alimente un environnement réel”.

- Trigger : `push` sur `main`
- Steps :
  - → `tests`
  - → `build` image
  - → `push :${{ github.sha }}`
  - → `deploy` sur Azure staging

✅ Intégration continue
✅ Artefact immuable (sha)
✅ Déploiement continu

Workflow complet - 3 Release

Objectif : “contractualiser une `release` et déployer en production”.

- Trigger : `push` sur tags `v*.*.*`
- Steps
  - → `tests`
  - → `build` image
  - → `push :v*.*.*`
  - → `deploy` sur Azure production
- Tags : `v*.*.*`
- Règles : ne jamais re-publier une version

✅ Versioning produit

Workflow complet - 4 Deploy (staging/prod)

On va utiliser GitHub Environments pour séparer la configuration et les secrets :

- `staging` : déploiement automatique
- `production` : approbation manuelle avant exécution

Workflow complet

Developer commit
↓
PR sur main
↓
PR CI (npm ci + tests)
↓
merge allowed / blocked

merge allowed : Push sur main
↓
- CI main : tests
- build image
- push (:sha + :latest)

↓
Registry (Managée ou Docker Hub)
↓
Deploy to Azure-STAGING (using immutable :sha image)

Git tag vX.Y.Z
↓
CI release
push (:vX.Y.Z [+ :sha])
↓
Deploy to Azure-PRODUCTION (using :vX.Y.Z)
↓
Manual approval via GitHub Environment

CI/CD

Les erreurs à éviter

- Pousser une image sans tests
- Utiliser `latest` en prod
- Re-publier un tag `vX.Y.Z`
- Confondre variables CI et variables runtime :
  - CI : tag, image name...
  - Runtime : `DB_HOST`, `DB_PASSWORD`...
- Secrets “committés”

Récap express

Ressources utiles

CI/CD & GitHub Actions

Azure & GitHub Actions

Concepts Git

Semantic Versioning (SemVer)